(2)

	s/e	frequency	Total cost
Algorithm Sum(A, n)			
{			
S = 0;	1	1	1
for(i=0; i<n; i++)	1	n+1	n+1
{			n
S = S + A[i];	1	n	
}			1
return S;	1	1	
}			So 2n+3

for(i=0; i<n; i++)

Eg:- n=5

i=0 ; 0<5 ✓
i=1 ; 1<5 ✓
i=2 ; 2<5 ✓
i=3 ; 3<5 ✓
i=4 ; 4<5 ✓
i=5 ; 5<5 ✗

n times true condition & 1 time false condition.

for return & Assignment Stub count is 1

2n+3
⇒ O(n)

(3)

	S/e	frequency	Total
Algorithm Sum(A, n)	—	—	—
{	—	—	—
S = 0.0;	1	1	1
for (i = 1; i <= n; i++)	1	n+1	n+1
{ S += a[i]; }	1	n	n
return S;	1	1	1
}			2n+3

for (i = 1 to n)

Eg:- n = 5

i = 1, 1 <= 5 ✓
i = 2, 2 <= 5 ✓
i = 3, 3 <= 5 ✓
i = 4, 4 <= 5 ✓
i = 5, 5 <= 5 ✓
i = 6, 6 <= 5 ✗ → 1 time Exit = 1

= 5 times true ⇒ n

So n+1

O(n)

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A Constituent unit of MAHE, Manipal)

Find the Time complexity using Step count method.

	s/e	frequency	Total cost
Void transpose (int **a, int n)	0	—	—
{	0	—	—
for (int i=0; i<n; i++)	1	n+1	n+1
for (int j=i+1; j<n; j++)	1	n(n+1)/2	n(n+1)/2
swap (a[i][j], a[j][i]);	1	n(n-1)/2	n(n-1)/2
}	0	—	—

① for (int i=0; i<n; i++)

how many times this for loop runs
i=0 to n-1 (b/3 i<n)
how many numbers from 0 to n-1 => (n)
for Eg: 0 to 4 => 0, 1, 2, 3, 4 => 5
how many numbs from 1 to n-1 => n-1
Eg: 1 to 4 => 1, 2, 3, 4 => 4

then 1 more time for checking the last exit condition so it is (n+1)

② for the 2nd for loop

for (int j = i+1 ; j<n; j++)

This is the inner for loop so consider outer loop as well

So whenever

i=0, j = 0+1 till n-1

i=0, j = 1 to n-1

⇓

so how many numbers so n-1 numbers

so n-1 times.

& 1 more time for last condition checking

so n-1+1 ⇒ n times.

i=1, • j = 1+1 till n-1.

j = 2 to n-1

⇓

so how many numbers ⇒ n-2 numbers

so n-2 times

& 1 more time for last condition

n-2+1 ⇒ n-1 times

i = n-1 ; j = n-1+1 = n to n-1 ⇒ 0

& 1 more time for last condition.

so 0+1 = 1 times

That is

if $i = 0$, $j = 1$ to $n-1 \Rightarrow (n-1)+1 \Rightarrow n$

$i = 1$, $j = 2$ to $n-1 \Rightarrow (n-2)+1 \Rightarrow n-1$

$i = n-1$, $j = n-1+1 = n$ to $n-1 \Rightarrow (n-1+1) \Rightarrow 1$

$\therefore 1 + 2 + 3 + \cdots + n$

$= n(n+1)/2$

3rd Statement

swap (a[i][j], a[j][i])

whenever $i = 0$, $j = 1$ to $n-1 \Rightarrow (n-1)$ times.

$i = 1$, $j = 2$ to $n-1 \Rightarrow (n-2)$ times

$i = n-1$, $j = n$ to $n-1 \Rightarrow 0$ times

$\Rightarrow 0 + 1 + \cdots + (n-1) \Rightarrow \frac{n(n+1)}{2} \Rightarrow \boxed{\frac{(n-1)(n)}{2}}$

So Total cost is

$$\frac{(n+1)}{1} + \frac{n(n+1)}{2} + \frac{n(n-1)}{2}$$

$$\frac{(n+1)}{1} + \left(\frac{n^2+n}{2}\right) + \left(\frac{n^2-n}{2}\right)$$

$$\frac{2n+2+n^2+n+n^2-n}{2}$$

$$\frac{2n^2+2n+2}{2}$$

$$\frac{2(n^2+n+1)}{2}$$

$$= O(n^2)$$

$n = 5$

0 to $n-1$ $\Rightarrow$ 0 to 4 = $5^{-}(n)$

1 to $n-1$ $\Rightarrow$ 1 to 4 = 4 $(n-1)$

2 to $n-1$ $\Rightarrow$ 2 to 4 = 3 $(n-2)$

$\vdots$

n to $n-1$ $\Rightarrow$ n to 4 $\Rightarrow$ $\underline{\underline{0}}^{(n-n)}$

1. int n; // assume n as power of 2
   while (n > 1) {
   n = n / 2;
   }

   Step Count Analysis
   We assume that:

- n is a power of 2, i.e., n = 2^k, where k≥k≥1.
- We want to count the number of steps it takes for the loop to terminate.

   Step Count Breakdown

1. **Initialization Step**:
   int n; (Assume n is initialized to some power of 2.)
   This is a **constant-time operation** and does not depend on n.
   Step count: 1.

2. **Loop Condition Check** (while (n > 1)):
   Each iteration, this condition is checked once.
   The number of checks depends on how many times the loop runs before n becomes 1.
   Let's denote the total number of iterations as **m**.

3. **Division Step** (n = n / 2):
   In each iteration, the value of n is divided by 2.

   Since n is a power of 2, i.e., n = 2^k, it takes $\log_2(n)$ divisions to reach 1

   Specifically, if n = 2^k, the number of divisions (and therefore the number of iterations) is it takes $\log_2(n)$

4. **Termination Step**:
   When $n == 1$, the loop ends. No further division happens after that.

Total Number of Steps

- **Step 1 (initialization)**: 1 step.
- **Step 2 (loop iterations)**: The loop runs $\log_2(n)$ times.
- **Step 3 (loop checks)**: The loop checks one more time after the last division, so there are $\log_2(n) + 1$ condition checks.
- **Step 4 (division assignments)**: The division assignment happens exactly $\log_2(n)$ times.

Thus, the total step count $T(n)$ is:

$$T(n) = 1 + (\log_2(n) + 1) + \log_2(n)$$

$$T(n) = 1 + \log_2(n) + 1 + \log_2(n)$$

$$T(n) = 2\log_2(n) + 2$$

Conclusion

Using step-count analysis, the total number of steps for the loop is:

$$T(n) = 2\log_2(n) + 2$$

This shows the complexity in terms of the number of steps is **O(log n)**.

(1) int n; //Assume n as power of 2.
while (n>1)
n= n/2

Example :- int n=16;
while (n>1)
n= n/2;

(1) Initialization int n= 16.
Step count = 1

(2) while n>1 is checked
16>1 is true → 1 step
operation= n= n/2 => n= 16/2 = 8
1 division step
} I-st iteration

(3) while n>1, 8>1 checked → 1 step
operation n= 8/2 => 4.
1 division step
} II-nd iteration

(4) 3rd iteration

$4 > 1$ true → 1 step

$n = n/2 = 4/2 = 2$

(5) 4th iteration

$2 > 1$ → true → 1 step

$n = n/2 = n = 1$

(6) Final check

$1 > 1$ False → 1 step loop ends

So total steps

Initialization = 1

Condition (5 checks) = 4 true + 1 false.

Division (4 division) = 4.

$\therefore \quad 1 + 5 + 4 = 10$

For $n = 16$ (which is 2^4)

$T(n) = 2 \log_2 n + 2$

$= 2 \log_2 16 + 2$

$= 2 \times 4 + 2 = 10$